

Fecha: 26 de Septiembre del 2026

CNO 4: CIBERSEGURIDAD

ACTIVIDAD 10 LAB: TRAVERSAL SEQUENCES STRIPPED WITH SUPERFLUOUS URL-DECODE

Documentación

Maestro: Servando López Contreras

Alumno: Quetzal Olin Rodríguez Jiménez

Contenido

OBJETIVO	2
AMBEINTE DE LABORATORIO	3
ANÁLISIS DE LA VULNERABILIDAD Y FORMA DE RESOLUCIÓN	6
COMANDOS UTILIZADOS	7
RESULTADOS	8
CONCLUSIONES	8

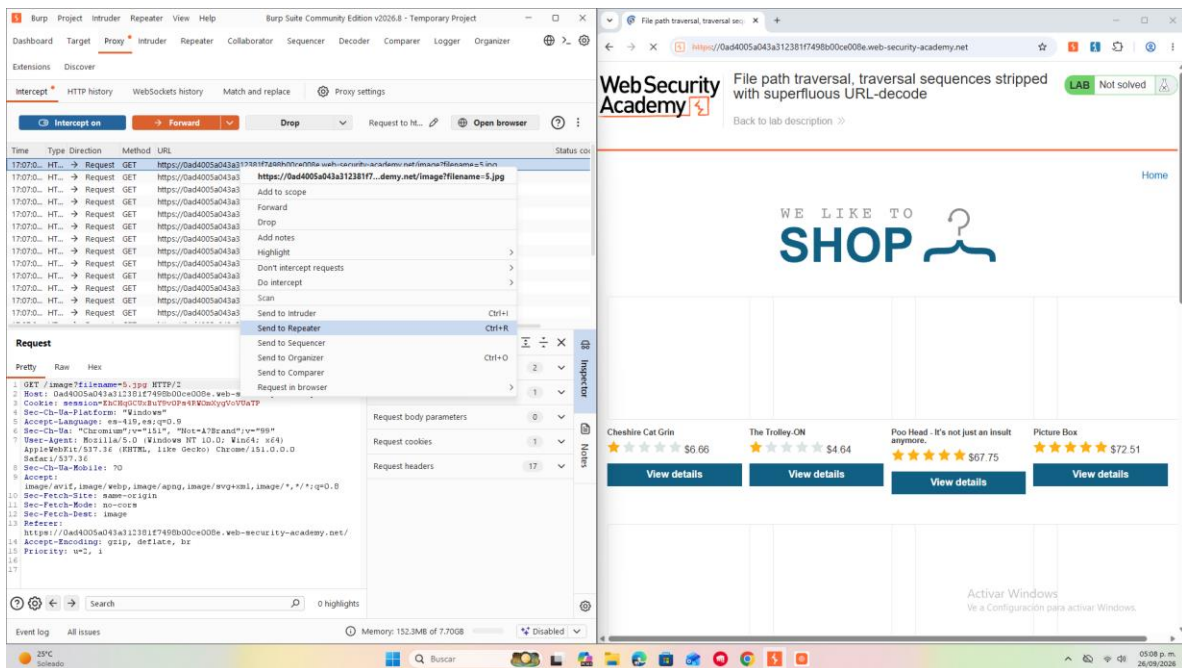
OBJETIVO

Actualizar la guía del laboratorio **FILE PATH TRAVERSAL — TRAVERSAL SEQUENCES STRIPPED WITH SUPERFLUOUS URL-DECODE** para que cubra técnicas de bypass basadas en **secuencias anidadas y codificación URL múltiple**. El objetivo sigue siendo explotar una sanitización no recursiva y/o un proceso de decodificación excesiva para leer archivos sensibles como **/etc/passwd**.

AMBEINTE DE LABORATORIO

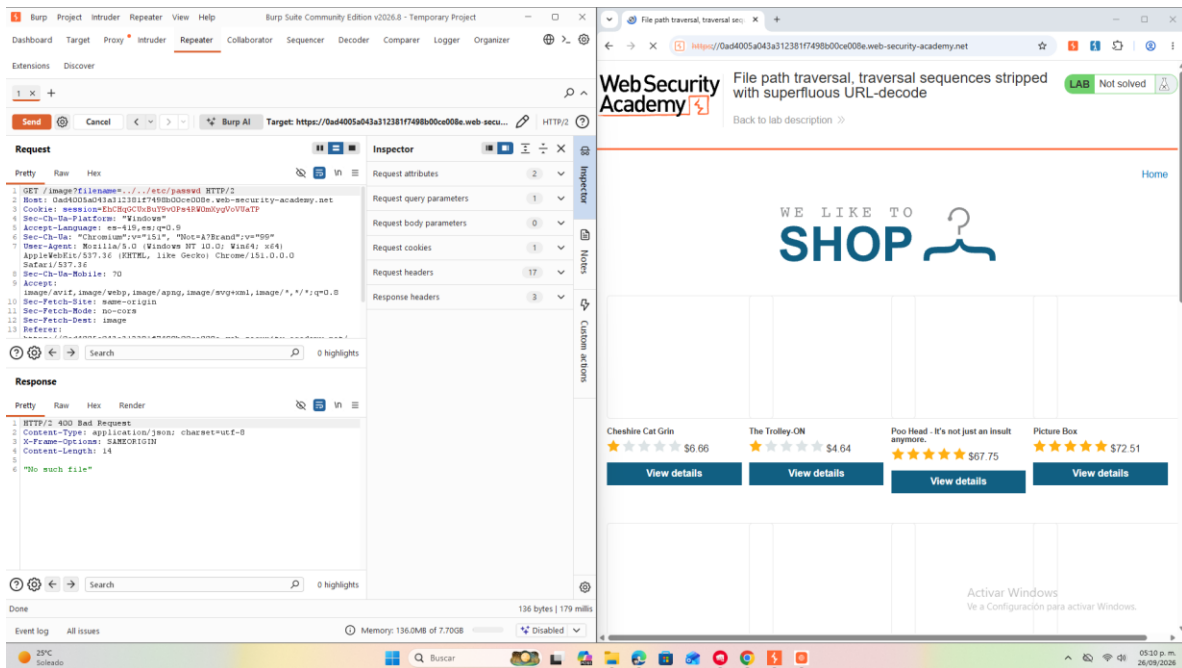
Identificar la petición

Localizar la petición GET que contiene `filename=blog.png` o similar.



Probar secuencias simples

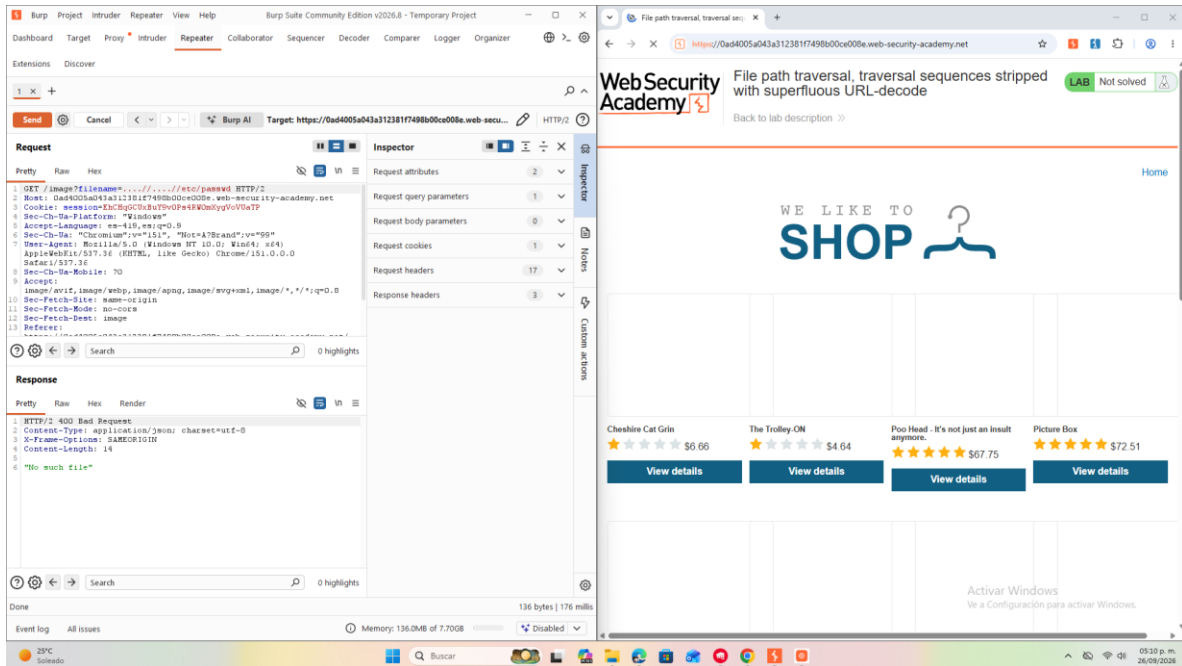
Reemplazar `filename` por `../../../../etc/passwd` o `/etc/passwd`.



Probar secuencias anidadas

Enviar filename=../../../../etc/passwd.

Resultado esperado: si el filtro es no recursivo, la petición resuelve y devuelve el contenido de /etc/passwd.



Probar bypass con codificación URL simple

Enviar `filename=%2e%2e%2f%2e%2e%2fetc%2fpasswd`.

Si el servidor decodifica una vez y luego aplica el filtro, puede bloquearlo; si decodifica de forma inadecuada, puede permitirlo.

The screenshot shows Burp Suite on the left and a web browser on the right. In Burp Suite, the 'Repeater' tab is active, showing a request to `https://0ad4005a043a12381f7498b00ce008e.web-security-academy.net`. The request is an HTTP/2 GET for `/image?filename=252e%252e%252f%252e%252e%252fetc%252fpasswd`. The response is an HTTP/2 400 Bad Request. The browser on the right shows the 'Web Security Academy' lab page titled 'File path traversal, traversal sequences stripped with superfluous URL-decode'. The page displays a 'WE LIKE TO SHOP' banner and a list of products: 'Cheshire Cat Grin' (\$6.66), 'The Trolley ON' (\$4.64), 'Poo Head - It's not just an insult anymore.' (\$67.75), and 'Picture Box' (\$72.51). A 'LAB Not solved' badge is visible in the top right corner of the browser window.

Probar bypass con doble codificación (superfluous URL-decode)

Enviar **doble codificado**: `filename=%252e%252e%252f%252e%252e%252fetc%252fpasswd`.

The screenshot shows Burp Suite on the left and a web browser on the right. In Burp Suite, the 'Repeater' tab is active, showing a request to `https://0ad4005a043a12381f7498b00ce008e.web-security-academy.net`. The request is an HTTP/2 GET for `/image?filename=252e%252e%252f%252e%252e%252fetc%252fpasswd`. The response is an HTTP/2 200 OK. The browser on the right shows the 'Web Security Academy' lab page titled 'File path traversal, traversal sequences stripped with superfluous URL-decode'. The page displays a 'WE LIKE TO SHOP' banner and a list of products: 'Cheshire Cat Grin' (\$6.66), 'The Trolley ON' (\$4.64), 'Poo Head - It's not just an insult anymore.' (\$67.75), and 'Picture Box' (\$72.51). A 'LAB Not solved' badge is visible in the top right corner of the browser window.

ANÁLISIS DE LA VULNERABILIDAD Y FORMA DE RESOLUCIÓN

Comportamiento inseguro observado

- El servidor aplica **reemplazos simples** para `../` o realiza una única pasada de decodificación.
- La sanitización **no es recursiva** y/o el servidor realiza **decodificaciones múltiples** en momentos distintos del pipeline, lo que permite que payloads especialmente contruidos se transformen en secuencias `../` válidas tras el procesamiento.

Por qué funcionan las técnicas

- **Secuencias anidadas:** `....//` contiene `..` y `//` de forma que, tras eliminar una porción no recursivamente, los fragmentos restantes se concatenan formando `../`.
- **Doble codificación:** si el servidor decodifica más de una vez o decodifica en etapas, una entrada doblemente codificada puede evadir la detección inicial y convertirse en `../` después de una decodificación intermedia.

Mitigaciones recomendadas

- **Canonicalizar rutas** usando funciones nativas del sistema (por ejemplo `realpath`, `Path.resolve`) y comparar que la ruta resultante esté dentro del **directorio base autorizado**.
- **No confiar en reemplazos de cadenas;** evitar sanitizaciones basadas solo en `replace("../", "")`.

COMANDOS UTILIZADOS

Burp Suite – Proxy (Intercept ON): Se activó la opción *Intercept on* para capturar las peticiones HTTP generadas por el navegador.

Forward: Se utilizó el botón *Forward* para permitir que la página cargara después de interceptar las solicitudes.

Observación de tráfico GET: Se identificaron múltiples peticiones de tipo **GET** hacia recursos estáticos.

Prueba de acceso a directorios: Se intentó navegar hacia el directorio raíz regresando niveles (`../`) para comprobar la estructura de directorios y posibles accesos indebidos.

Send to Repeater: Envío de la solicitud del parámetro filename al entorno de pruebas repetitivas de Burp Suite.

Manipulación de Parámetro GET (filename): Modificación del valor por secuencias anidadas no recursivas (....//) para evadir los filtros de validación de entradas.

RESULTADOS

Se identificó que la aplicación web implementa una sanitización débil basada en expresiones regulares o reemplazos simples que operan de forma no recursiva.

Se comprobó que el uso de payloads anidados (....//) permite superar el bloqueo de secuencias simples de tipo ../.

Con filtros no recursivos y/o decodificación superflua, payloads como
....//....//etc/passwd o variantes doblemente codificadas pueden permitir la lectura de
/etc/passwd.

Burp Repeater acelera la iteración de payloads y la confirmación del bypass.

CONCLUSIONES

Debilidad de los filtros basados en reemplazos únicos: Las funciones de sanitización que eliminan cadenas prohibidas (como ../) sin aplicar un bucle recursivo hasta que no queden coincidencias son altamente vulnerables a técnicas de anidación de caracteres.

Las sanitizaciones basadas en un único reemplazo son insuficientes.

La combinación de **secuencias anidadas** y **codificación URL múltiple** es una técnica efectiva contra filtros débiles.

La defensa robusta requiere **canonicalización**, **listas blancas**, y **validación post-normalización**.